BCO701 IoT Lab — Exam Study Guide

All 12 experiments, checked and corrected. For each one: what to wire, the code to write, what you should see, what goes wrong, and short viva answers.

1. Which IDE to use

Use Arduino IDE 2.x (the same IDE you used last time). Every experiment in this manual is an Arduino sketch — even the ESP8266 (NodeMCU) and ESP32 programs are written in the Arduino language, you just add the board package. It is also what the lab computers almost certainly have, and what the examiner expects to see. Download from arduino.cc/en/software if it isn't installed.

To practise tonight without hardware, two free browser simulators help:

- **Tinkercad Circuits** (tinkercad.com) — Arduino Uno with LED, push button, photoresistor, relay, ultrasonic sensor, DC motor + L293D driver. Good for Exp 2, 3, 5, 9, 10. It has a TMP36 instead of LM35 (TMP36 formula: °C = (V − 0.5) × 100), so Exp 4 needs that tweak there.
- **Wokwi** (wokwi.com) — simulates ESP32 *with internet* (use SSID `Wokwi-GUEST`, empty password). You can run Exp 1 (MQTT) live there and Exp 6/8/11 by porting to ESP32 (swap `ESP8266WiFi.h`→`WiFi.h`, `ESP8266HTTPClient.h`→`HTTPClient.h`). It also has DHT22 and HC-SR04 parts.

One-time setup in Arduino IDE

- **File → Preferences → Additional boards manager URLs**, paste both (comma-separated):
  `http://arduino.esp8266.com/stable/package_esp8266com_index.json`
  `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
- **Tools → Board → Boards Manager**: install **esp8266** (by ESP8266 Community) and **esp32** (by Espressif).
- **Sketch → Include Library → Manage Libraries**: install the libraries in the table below.
- No COM port showing? Install the USB driver: **CH340** (most cheap NodeMCU/Uno clones) or **CP2102** (most ESP32 DevKits). Also try another USB cable — many are charge-only.

Exp	Board (Tools → Board)	Libraries to install	Baud (Serial Monitor)
1	ESP32 Dev Module	PubSubClient (Nick O'Leary)	115200
2, 3, 4, 5, 9, 10	Arduino Uno	none	9600
6 (option A)	Arduino Uno + ESP-01	none (SoftwareSerial is built in)	9600
6 (option B), 7, 8, 11	NodeMCU 1.0 (ESP-12E Module)	Exp 7: ThingSpeak (MathWorks). Others: none — ESP8266WiFi and ESP8266HTTPClient come with the board package	115200
12	Arduino Uno	DHT sensor library (Adafruit) + Adafruit Unified Sensor	9600

Things that will cost you marks on exam day if you don't know them

- ESP8266/ESP32 only join **2.4 GHz** Wi-Fi, and cannot log in to college Wi-Fi with a login page. Use your **phone hotspot** set to 2.4 GHz (on iPhone turn on 'Maximise Compatibility'). SSID and password are case-sensitive.
- Serial Monitor baud must match `Serial.begin()` — garbage characters mean wrong baud.
- Upload fails on ESP board: hold the **BOOT** button while 'Connecting...' appears, then release.
- Upload fails on Uno in Exp 6 option A: unplug the ESP wire from pin 0/1 during upload (my version uses pins 2/3 to avoid this).

2. What I found wrong in the manual

I compiled every program (with stand-in Arduino headers — the board packages can't be downloaded in my workspace, so this checks syntax and types, not real hardware). All corrected programs below

compile cleanly.

Exp	Problem in manual	Severity	Fix used in this guide
1	Fixed client ID 'ESP32Client' on a public broker gets kicked when someone else uses it; no reconnect in loop()	Works sometimes	Random client ID + reconnect()
2	Lists only a 10 kΩ resistor; LED needs its own 220 Ω	Wiring	Code fine; add 220 Ω
3	Objective says 'indicate brightness or darkness' but code only prints	Minor	Added threshold + LED
4	Output table shows 150 °C/200 °C — LM35 max is 150 °C, 410 → 200 °C is impossible	Table only	Code correct as is
5	**Does not compile.** Lines merged so the digitalWrite() calls are inside comments and BUTTON_PIN is never defined	**Broken**	Rewritten properly
6	Text wires ESP to pins 0/1 but code uses pins 2/3; HTTP request missing 'HTTP/1.1' and Host header; println adds 2 bytes so CIPSEND length is wrong; ESP-01 default baud is usually 115200, not 9600; sample output is invented (ThingSpeak returns an entry number, not HTML)	**Likely fails**	Fixed AT version + an easier NodeMCU version (POST to send, GET to receive)
7	LM35 on 3.3 V is below its 4 V minimum supply; channel number left as a placeholder (won't compile)	Wiring	Power LM35 from VIN (5 V); output stays < 1.5 V so A0 is safe
8	FirebaseArduino library is abandoned and no longer works with Firebase; no wait for Wi-Fi	**Broken**	Firebase REST API over HTTPS — no extra library
9	If no echo, pulseIn returns 0 → distance 0 → false 'Intruder'; buzzer/LED listed but unused	Logic bug	Timeout + check, LED and buzzer added
10	Reverses motor with no stop in between; common GND not mentioned	Minor	Added stop; wiring note
11	**Only placeholder code.** Google Cloud IoT Core was shut down by Google in August 2023, so it cannot be done as written	**Missing**	Full program sending distance to Firebase (a Google cloud platform)
12	No check for failed read (prints 'nan')	Minor	isnan() check

Experiment 1 — Study of IoT Protocols + MQTT LED control

Aim: To study 6LoWPAN, IPv4/IPv6, Wi-Fi, Bluetooth and MQTT, and implement MQTT publish/subscribe to control an LED on ESP32.

Part A — Protocol theory (write this as a comparison table in the exam)

Protocol	Layer	Key points
6LoWPAN	Adaptation (between IPv6 and IEEE 802.15.4 MAC)	IPv6 over Low-power Wireless Personal Area Networks. Compresses the 40-byte IPv6 header to as little as 2–4 bytes (IPHC) and fragments packets to fit the 127-byte 802.15.4 frame. Lets tiny battery sensors have IP addresses. Used in Thread, smart meters.
IPv4	Network	32-bit address (~4.3 billion), dotted decimal 192.168.1.10. Address space exhausted; relies on NAT. Header 20–60 bytes.
IPv6	Network	128-bit address (3.4×10^{38}), hex colon form 2001:db8::1. Fixed 40-byte header, no NAT needed, built-in auto-configuration (SLAAC), IPsec support. Preferred for IoT because billions of devices each need an address.
Wi-Fi (IEEE 802.11)	PHY + MAC	2.4/5 GHz, range ~50–100 m indoors, high data rate (Mbps–Gbps), high power. Direct internet via router. ESP8266/ESP32.
Bluetooth / BLE (IEEE 802.15.1)	PHY + MAC	2.4 GHz, range ~10 m (BLE up to ~100 m), BLE uses very low power, 1–2 Mbps. Wearables, beacons, phone-to-device. ESP32 has BLE built in.
MQTT	Application (over TCP, port 1883; 8883 with TLS)	Lightweight publish/subscribe. Clients publish to a *topic*; a *broker* forwards to all subscribers. 2-byte minimum header. QoS 0/1/2, retained messages, Last Will.

Part B — MQTT practical

Components: ESP32 DevKit, USB cable, (optional) LED + 220 Ω on GPIO 2 — the on-board LED is already on GPIO 2.

Changed from the manual

- Random client ID — the public broker disconnects a client when another connects with the same ID, and many students use 'ESP32Client'.
- reconnect() in loop() so it recovers if the broker drops the connection.
- Publishes a status message back, so you can show both publish *and* subscribe.

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid       = "Your_SSID";        // 2.4 GHz hotspot
const char* password   = "Your_PASSWORD";
const char* mqttServer = "broker.hivemq.com";
const int   mqttPort   = 1883;
const char* topicSub   = "iot/led";           // use a unique topic when testing
const char* topicPub   = "iot/led/status";

WiFiClient espClient;
PubSubClient client(espClient);
const int ledPin = 2;                         // on-board LED of ESP32 DevKit

void callback(char* topic, byte* payload, unsigned int length) {
  String message = "";
  for (unsigned int i = 0; i < length; i++) {
    message += (char)payload[i];
  }
  message.trim();
  Serial.print("Message on ");
  Serial.print(topic);
  Serial.print(": ");
  Serial.println(message);
```

```
  if (message == "ON") {
    digitalWrite(ledPin, HIGH);
    client.publish(topicPub, "LED ON");
  } else if (message == "OFF") {
    digitalWrite(ledPin, LOW);
    client.publish(topicPub, "LED OFF");
  }
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connecting to MQTT broker...");
    String clientId = "ESP32-" + String(random(0xffff), HEX);  // unique ID
    if (client.connect(clientId.c_str())) {
      Serial.println("connected");
      client.subscribe(topicSub);
    } else {
      Serial.print("failed, rc=");
      Serial.println(client.state());
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);

  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");

  client.setServer(mqttServer, mqttPort);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    reconnect();                // re-connects if broker drops us
  }
  client.loop();                // processes incoming messages
}
```

How to send ON/OFF (pick one)

- **HiveMQ browser client:** search 'HiveMQ websocket client', host broker.hivemq.com, port 8884, Connect → Publish topic `iot/led`, message `ON`. Subscribe to `iot/led/status` to see the reply.
- **MQTTX** desktop app or **Node-RED** (mqtt out node → broker.hivemq.com:1883).
- When testing tonight, change the topic to something unique like `bco701/yourUSN/led` so strangers on the public broker can't toggle your LED. In the exam answer you can keep `iot/led`.

Expected Serial Monitor output

```
Connecting to WiFi.....
WiFi connected
Connecting to MQTT broker...connected
Message on iot/led: ON
Message on iot/led: OFF
```

If it doesn't work

- rc=-2 → network can't reach broker (college firewall may block port 1883 — use hotspot).
- Message arrives but LED doesn't change → you sent 'on' in lowercase; the check is case-sensitive.
- 'PubSubClient.h: No such file' → install PubSubClient from Library Manager.

Viva — quick answers

Role of 6LoWPAN?

An adaptation layer that carries IPv6 packets over low-power 802.15.4 radios by header compression and fragmentation, giving tiny devices end-to-end IP connectivity.

Why IPv6 over IPv4 for IoT?

Huge address space (128-bit) so every device gets a unique address without NAT, plus auto-configuration, simpler header and built-in security.

Wi-Fi vs Bluetooth?

Wi-Fi: longer range (~100 m), higher data rate, higher power, direct internet. Bluetooth/BLE: short range (~10 m), lower rate, very low power, device-to-device.

What is MQTT / publish-subscribe?

A lightweight messaging protocol over TCP. Publishers send to topics on a broker; the broker delivers to every subscriber of that topic. Sender and receiver never talk directly (decoupled).

MQTT QoS levels?

QoS 0: at most once (fire and forget). QoS 1: at least once (acknowledged, may duplicate). QoS 2: exactly once (4-step handshake, slowest).

Lightweight alternatives to MQTT?

CoAP (REST over UDP), AMQP, XMPP, DDS, MQTT-SN (for non-TCP sensor networks).

How does 6LoWPAN compress IPv6 headers?

IPHC elides fields that can be derived from the link layer (addresses from MAC address, payload length from frame length) or that are common (version, traffic class, flow label), shrinking 40 bytes to ~2–4.

Experiment 2 — Controlling an LED with a Push Button

Aim: To control an LED using a push button with Arduino UNO.

Components: Arduino Uno, push button, LED, 220 Ω resistor (LED), 10 kΩ resistor (pull-down), breadboard, jumper wires.

Changed from the manual

- Code unchanged — it's correct. Added the 220 Ω LED resistor the component list forgot.

Connections

From	To
Button leg 1	5V
Button leg 2	Pin 4, and through 10 kΩ to GND (pull-down)
LED anode (long leg)	Pin 13 through 220 Ω
LED cathode (short leg)	GND

Program

```
const int buttonPin = 4;     // Push button on digital pin 4 (10k pull-down to GND)
const int ledPin = 13;       // LED on pin 13 (via 220 ohm resistor)

int buttonState = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT);
}

void loop() {
  buttonState = digitalRead(buttonPin);
  if (buttonState == HIGH) {
    digitalWrite(ledPin, HIGH);   // pressed -> LED ON
  } else {
    digitalWrite(ledPin, LOW);    // released -> LED OFF
  }
}
```

How it works

- The pull-down resistor holds pin 4 at LOW (0) when the button is open; pressing connects it to 5 V → HIGH (1).
- Without the pull-down the pin 'floats' and reads random values — the LED flickers.
- Shortcut: `pinMode(buttonPin, INPUT_PULLUP)` uses the internal resistor; then wire the button to GND and the logic inverts (pressed = LOW).

If it doesn't work

- LED always on → button rotated 90° on the breadboard (its legs on the same side are already connected). Turn it.
- Pin 13 also drives the on-board 'L' LED, so you'll see that blink even without an external LED.

Viva — quick answers

What is a pull-down resistor?
A resistor from the input pin to GND that keeps the pin at a defined LOW when the switch is open.

digitalRead() returns?
HIGH (1) or LOW (0).

Why the 220 Ω resistor?
Limits LED current to about (5 − 2)/220 ≈ 14 mA, protecting the LED and pin (pin max 40 mA, recommended 20 mA).

What is switch bouncing?
Mechanical contacts make and break several times in a few ms when pressed; fixed with a short delay or millis()-based debounce.

Experiment 3 — Detection of Light using Photoresistor (LDR)

Aim: To detect the intensity of light using an LDR and display the result using Arduino.

Components: Arduino Uno, LDR, 10 kΩ resistor, breadboard, jumper wires.

Changed from the manual

- Added a threshold and LED so the program actually 'indicates brightness or darkness' as the objective says. Removed pinMode(A0, INPUT) (not needed for analogRead).

Connections

From	To
LDR leg 1	5V
LDR leg 2	A0, and through 10 kΩ to GND
(optional) LED	Pin 13 — on-board LED works

Program

```
// LDR Light Detection Program
const int ldrPin = A0;      // junction of LDR and 10k resistor
const int ledPin = 13;      // indicator LED (on-board)
const int threshold = 400;  // below this = dark (tune after seeing values)
int ldrValue = 0;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  ldrValue = analogRead(ldrPin);          // 0 - 1023
  Serial.print("Light Intensity: ");
  Serial.print(ldrValue);

  if (ldrValue < threshold) {
    Serial.println("  -> DARK");
    digitalWrite(ledPin, HIGH);           // switch light ON in darkness
  } else {
    Serial.println("  -> BRIGHT");
    digitalWrite(ledPin, LOW);
  }
  delay(1000);
}
```

How it works

- LDR + 10 kΩ form a voltage divider. Bright light → LDR resistance drops (to ~1 kΩ) → more voltage at A0 → higher reading.
- Dark → resistance rises (to ~1 MΩ) → A0 voltage falls → low reading.
- analogRead() uses a 10-bit ADC: 0–1023 for 0–5 V. Each step ≈ 4.9 mV.
- Below the threshold the LED switches on — this is how an automatic street light works.

Expected Serial Monitor output

```
Light Intensity: 812  -> BRIGHT
Light Intensity: 790  -> BRIGHT
Light Intensity: 214  -> DARK
Light Intensity: 38  -> DARK
```

If it doesn't work

- Readings go the opposite way (higher in dark)? You swapped LDR and resistor — either swap back or flip the comparison.
- Values stuck near 0 or 1023 → A0 wire not at the junction between LDR and resistor.

Viva — quick answers

Working principle of LDR?
Photoconductivity: light frees charge carriers in cadmium sulphide, so resistance falls as light increases.

Range of analogRead()?
0 to 1023 (10-bit ADC on Uno).

Analog or digital sensor?
Analog — its output varies continuously.

Role of the 10 kΩ resistor?
Forms a voltage divider with the LDR so changing resistance becomes a changing voltage the ADC can read.

IoT use of LDR?
Automatic street lights, smart curtains, display brightness control, daylight-based energy saving.

Experiment 4 — Interfacing Temperature Sensor LM35 with Arduino

Aim: To interface LM35 with Arduino and display the temperature on the Serial Monitor.

Components: Arduino Uno, LM35, breadboard, jumper wires.

Changed from the manual

- Code is correct. Ignore the manual's output table rows for 150/200 °C — LM35 tops out at 150 °C; real room readings are about 55–65 counts (27–32 °C).

Connections

From	To
LM35 pin 1 (+Vs)	5V
LM35 pin 2 (Vout)	A0
LM35 pin 3 (GND)	GND
Orientation	Flat face towards you, legs down: left = +Vs, middle = Vout, right = GND

Program

```
// LM35 Temperature Sensor Interfacing
const int lm35Pin = A0;
float tempC;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int analogValue = analogRead(lm35Pin);              // 0 - 1023
  tempC = (analogValue * 5.0 * 100.0) / 1024.0;       // 10 mV per degree C

  Serial.print("Temperature: ");
  Serial.print(tempC);
  Serial.println(" C");

  delay(1000);
}
```

How it works

- LM35 outputs 10 mV per °C (0 V at 0 °C). At 28 °C → 0.28 V.
- Voltage = reading × 5.0 / 1024. Temperature = voltage / 0.01 = voltage × 100.
- So °C = reading × 5.0 × 100 / 1024 ≈ reading × 0.488. Each ADC step ≈ 0.49 °C.

Expected Serial Monitor output

```
Temperature: 28.32 C
Temperature: 28.81 C
Temperature: 28.32 C
```

If it doesn't work

- Sensor gets **hot** to touch → it's in backwards. Unplug immediately and flip it.
- Reads ~500 °C → Vout and +Vs swapped. Reads 0 → output wire not on A0.
- Readings jump by ~0.5 °C — that's normal (ADC resolution).

Viva — quick answers

Sensitivity of LM35?
10 mV/°C, accuracy about ±0.5 °C at 25 °C. Range −55 to 150 °C.

How is temperature calculated?
°C = analogRead × 5.0 × 100 / 1024 (convert reading to volts, then divide by 10 mV).

Resolution of Arduino ADC?

10 bits → 1024 levels → 5 V / 1024 ≈ 4.88 mV per step.

Can LM35 measure negative temperatures?

Yes, down to −55 °C, but only with a negative supply/offset circuit; in this basic single-supply setup it reads 2 °C and above.

Why multiply by 100?

Because 1 °C = 10 mV = 0.01 V, so dividing volts by 0.01 is the same as multiplying by 100.

Experiment 5 — Interfacing a Relay with Arduino (motor ON with push button)

Aim: To turn ON a motor through a relay when a push button is pressed, using Arduino Uno.

Components: Arduino Uno, 5 V relay module, push button, 10 kΩ resistor, DC motor/bulb, external supply (battery) for the motor, jumper wires.

Changed from the manual

- **The manual's code doesn't compile**: the line breaks were lost, so '#define BUTTON_PIN 2' and both digitalWrite() calls ended up inside // comments. Even if it compiled, the relay would never switch. Rewritten below.
- Removed the 1-second delay that made the button feel unresponsive; relay is forced OFF at start-up.

Connections

From	To
Relay module VCC / GND	5V / GND
Relay module IN	Pin 8
Button	One leg to 5V, other leg to Pin 2 and via 10 kΩ to GND
Motor circuit	Battery + → relay COM; relay NO → motor; motor other wire → battery −

Program

```
#define RELAY_PIN 8    // relay module IN pin
#define BUTTON_PIN 2   // push button (10k pull-down)

void setup() {
  Serial.begin(9600);
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT);
  digitalWrite(RELAY_PIN, LOW);           // start with motor OFF
}

void loop() {
  int buttonState = digitalRead(BUTTON_PIN);

  if (buttonState == HIGH) {
    // Button pressed -> relay energised -> motor ON
    digitalWrite(RELAY_PIN, HIGH);
    Serial.println("Button pressed: Motor ON");
  } else {
    // Button released -> relay off -> motor OFF
    digitalWrite(RELAY_PIN, LOW);
    Serial.println("Button released: Motor OFF");
  }
  delay(200);
}
```

How it works

- The Arduino pin can't power a motor (max ~40 mA). It drives the relay coil (via the module's transistor), and the relay contacts switch the separate motor supply.
- COM–NO (normally open) closes when the relay is energised → motor runs. COM–NC is closed when off.
- The relay also isolates the Arduino from the motor's higher voltage/current and electrical noise.

Expected Serial Monitor output

```
Button released: Motor OFF
Button pressed: Motor ON
Button pressed: Motor ON
Button released: Motor OFF
```

If it doesn't work

- Relay ON when button is NOT pressed → your module is **active-LOW** (common on blue modules). Swap HIGH and LOW in the two digitalWrite() lines.
- Relay clicks but motor doesn't run → motor wired to NC instead of NO, or the battery is flat.
- LED on relay module lights but no click → module needs 5 V, not 3.3 V.

Viva — quick answers

What is a relay?
An electrically operated switch: a small current through a coil creates a magnetic field that moves a contact, switching a separate high-power circuit.

NO vs NC?
Normally Open: connected to COM only when energised. Normally Closed: connected to COM when not energised.

Why use a relay instead of connecting the motor directly?
Arduino pins supply only ~20–40 mA at 5 V; relays switch large currents/voltages (even 230 V AC) and isolate the controller.

Purpose of the flyback diode on relay modules?
Absorbs the voltage spike produced when the coil is switched off, protecting the transistor and Arduino.

Experiment 6 — Send and Receive Data using HTTP

Aim: To develop an application to send and receive data with Arduino using HTTP requests (GET/POST) to a server (ThingSpeak).

Changed from the manual

- The manual's wiring text (ESP to pins 0/1) contradicts its code (SoftwareSerial pins 2/3). Use pins 2/3 — pins 0/1 block uploading.
- Its request line `GET /update?...\r\n` is missing `HTTP/1.1` and the `Host:` header that ThingSpeak requires.
- It sends the request with println(), which adds 2 extra bytes, so the length given to AT+CIPSEND is wrong.
- Most ESP-01 modules ship at 115200 baud — SoftwareSerial at 9600 won't talk to them until you change it (see tip below).
- The sample output (an HTML page) isn't what ThingSpeak returns — a successful update returns just the entry number, e.g. 23; `0` means rejected.

Before either option: create a ThingSpeak channel

- thingspeak.com → sign up (MathWorks account) → Channels → New Channel → tick Field 1 → Save.
- API Keys tab → copy **Write API Key** and **Read API Key**; note the **Channel ID**.
- Free accounts accept one update every **15 seconds** — faster updates return 0.

Option A — Arduino Uno + ESP-01 using AT commands (matches the manual)

From	To
ESP-01 VCC and CH_PD (EN)	**External 3.3 V** supply (e.g. AMS1117 module). The Uno's 3.3 V pin (~50 mA) is too weak — the #1 reason this experiment fails
ESP-01 GND	GND (common with Arduino)
ESP-01 TX	Arduino pin 2
ESP-01 RX	Arduino pin 3 through divider: pin 3 → 1 kΩ → ESP RX → 2 kΩ → GND (5 V → 3.3 V)

```
#include <SoftwareSerial.h>

SoftwareSerial esp(2, 3);   // RX = D2 (to ESP TX), TX = D3 (to ESP RX via divider)
String apiKey = "YOUR_WRITE_API_KEY";

void sendCommand(String cmd, int timeout) {
  esp.println(cmd);
  unsigned long start = millis();
  while (millis() - start < (unsigned long)timeout) {
    while (esp.available()) {
      Serial.write(esp.read());           // echo ESP reply to Serial Monitor
    }
  }
}

void setup() {
  Serial.begin(9600);
  esp.begin(9600);                        // ESP must be set to 9600 baud first
  sendCommand("AT", 2000);
  sendCommand("AT+CWMODE=1", 2000);       // station mode
  sendCommand("AT+CWJAP=\"Your_SSID\",\"Your_PASSWORD\"", 8000);
}

void loop() {
  int value = analogRead(A0);             // or a hard-coded value, e.g. 45

  sendCommand("AT+CIPSTART=\"TCP\",\"api.thingspeak.com\",80", 3000);

  String httpRequest = "GET /update?api_key=" + apiKey + "&field1=" + String(value) +
                       " HTTP/1.1\r\nHost: api.thingspeak.com\r\nConnection: close\r\n\r\n";

  sendCommand("AT+CIPSEND=" + String(httpRequest.length()), 2000);
  esp.print(httpRequest);                 // print (not println) so length matches
```

```
  unsigned long start = millis();         // read the server's response
  while (millis() - start < 5000) {
    while (esp.available()) Serial.write(esp.read());
  }
  sendCommand("AT+CIPCLOSE", 1000);
  delay(20000);                           // ThingSpeak free: >= 15 s between updates
}
```

Baud-rate fix

- If 'AT' gets no OK: change `esp.begin(9600)` to `esp.begin(115200)`, upload, and add `sendCommand("AT+UART_DEF=9600,8,1,0,0", 2000);` as the first command (old firmware: `AT+CIOBAUD=9600`). Run once, then set it back to 9600.

Expected output (Option A)

```
AT
OK
AT+CWMODE=1
OK
AT+CWJAP="MyHotspot","****"
WIFI CONNECTED
WIFI GOT IP
OK
AT+CIPSTART="TCP","api.thingspeak.com",80
CONNECT
OK
AT+CIPSEND=95
>
SEND OK
+IPD,...:HTTP/1.1 200 OK
...
23
CLOSED
```

Option B — NodeMCU (much easier, use this if the lab has NodeMCU)

No extra wiring — plug in the USB cable. Sends with **POST** and reads back with **GET**, so it covers both 'send and receive'. A0 is left floating, so the value will be random — that's fine, or connect any sensor.

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

const char* ssid = "Your_SSID";
const char* password = "Your_PASSWORD";
String writeKey = "YOUR_WRITE_API_KEY";
String readKey  = "YOUR_READ_API_KEY";
String channelId = "YOUR_CHANNEL_ID";

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
}

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    WiFiClient client;
    HTTPClient http;
    int value = analogRead(A0);

    // ---- SEND data: HTTP POST ----
    http.begin(client, "http://api.thingspeak.com/update");
    http.addHeader("Content-Type", "application/x-www-form-urlencoded");
    int code = http.POST("api_key=" + writeKey + "&field1=" + String(value));
    Serial.print("POST value ");
    Serial.print(value);
    Serial.print(" -> HTTP ");
    Serial.print(code);
    Serial.print(", entry id: ");
```

```
    Serial.println(http.getString());     // 0 means update rejected
    http.end();

    // ---- RECEIVE data: HTTP GET ----
    http.begin(client, "http://api.thingspeak.com/channels/" + channelId +
                       "/fields/1/last.txt?api_key=" + readKey);
    code = http.GET();
    Serial.print("GET -> HTTP ");
    Serial.print(code);
    Serial.print(", last value on server: ");
    Serial.println(http.getString());
    http.end();
  }
  delay(20000);
}
```

```
WiFi connected
POST value 512 -> HTTP 200, entry id: 24
GET -> HTTP 200, last value on server: 512
```

How it works

- HTTP is a request–response protocol over TCP (port 80). The client sends a request (method + path + headers), the server returns a status code (200 OK, 404 Not Found…) and a body.
- **GET** sends data in the URL query string; **POST** sends it in the request body.
- AT+CIPSTART opens the TCP connection, AT+CIPSEND=n tells the ESP n bytes are coming, then the raw HTTP text is sent.

Viva — quick answers

What is HTTP?
HyperText Transfer Protocol: a stateless, client–server, request–response application protocol over TCP, port 80 (443 for HTTPS).

GET vs POST?
GET puts data in the URL, is meant for retrieving, limited length, cached. POST puts data in the body, used for sending/creating data.

Default baud rate of ESP8266?
115200 on most current AT firmware (older modules 9600 or 57600).

Role of AT commands?
Text commands sent over UART to control the ESP8266 Wi-Fi module: set mode, join a network, open TCP connections, send data.

Explain AT+CIPSEND.
AT+CIPSEND=<length> tells the module how many bytes to send on the open connection; after the '>' prompt you send exactly that many bytes.

Why a voltage divider on ESP RX?
ESP8266 runs at 3.3 V; Arduino TX is 5 V. 1 kΩ/2 kΩ gives 5 × 2/3 ≈ 3.3 V.

Experiment 7 — Room Temperature to Cloud (LM35 + NodeMCU + ThingSpeak)

Aim: To measure room temperature using LM35 and post the value to the ThingSpeak cloud via NodeMCU ESP8266.

Components: NodeMCU ESP8266, LM35, breadboard, jumper wires, USB cable, Wi-Fi hotspot.

Changed from the manual

- LM35 powered from VIN (5 V) instead of 3.3 V — 3.3 V is below its rated minimum and gives wrong readings.
- Placeholder channel number replaced with a sample number (the original won't compile). Prints the error code on failure. Delay 20 s for safety.

Connections

From	To
LM35 +Vs	**VIN** (5 V from USB) — LM35 needs at least 4 V
LM35 Vout	A0
LM35 GND	GND

Program

```
#include <ESP8266WiFi.h>
#include <ThingSpeak.h>

const char* ssid = "Your_SSID";
const char* password = "Your_PASSWORD";

WiFiClient client;

unsigned long myChannelNumber = 1234567;      // replace with YOUR channel number
const char* myWriteAPIKey = "YOUR_WRITE_API_KEY";

const int lm35Pin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
  ThingSpeak.begin(client);
}

void loop() {
  int sensorValue = analogRead(lm35Pin);           // 0 - 1023 for 0 - 3.3 V
  float voltage = sensorValue * (3.3 / 1023.0);
  float temperatureC = voltage * 100.0;            // 10 mV per degree C

  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.println(" C");

  ThingSpeak.setField(1, temperatureC);
  int status = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
  if (status == 200) {
    Serial.println("Data sent to ThingSpeak.");
  } else {
    Serial.print("Error sending data. Code: ");
    Serial.println(status);
  }
  delay(20000);                                    // >= 15 s for free accounts
}
```

How it works

- NodeMCU's A0 reads 0–3.3 V as 0–1023 (the board has a divider; the bare chip is 0–1 V). So V = reading × 3.3/1023.
- LM35 output at room temperature is ~0.3 V — well within A0's range, so powering it from 5 V is safe.
- ThingSpeak.writeFields() makes an HTTP request to ThingSpeak and returns 200 on success.
- Open your channel → Private View to see the live chart.

Expected Serial Monitor output

```
Connecting to WiFi....
WiFi connected
Temperature: 28.71 C
Data sent to ThingSpeak.
```

If it doesn't work

- Compile error at 'YOUR_CHANNEL_NUMBER' → replace with your real channel number (digits only, no quotes).
- Error code -401 → posting faster than 15 s or wrong write key. Code 0/-301 → no internet.
- Temperature ~3× too high or noisy → check you used 3.3/1023, not 5.0/1024.

Viva — quick answers

Resolution of LM35?
10 mV/°C; with NodeMCU's ADC each step is 3.3/1023 ≈ 3.2 mV ≈ 0.32 °C.

Why 3.3 V on ESP8266?
The ESP8266 chip is a 3.3 V device; 5 V on its GPIO can damage it. (The LM35 itself still needs ≥4 V supply, so it's powered from VIN.)

What is ThingSpeak?
A MathWorks IoT cloud platform that stores sensor data in channels (up to 8 fields), plots it, and supports MATLAB analysis, via REST/MQTT APIs.

Maximum analog input of NodeMCU?
3.3 V on the A0 pin of the NodeMCU board (1.0 V on the bare ESP8266 ADC pin).

How often can we update ThingSpeak?
Once every 15 seconds on a free account.

Experiment 8 — Soil Moisture to Google Firebase

Aim: To measure soil moisture and post the sensed data to the Google Firebase Realtime Database using NodeMCU.

Components: NodeMCU ESP8266, soil moisture sensor (with module board), jumper wires.

Changed from the manual

- The FirebaseArduino library in the manual is abandoned and no longer connects to Firebase. This version uses Firebase's REST API with the built-in ESP8266HTTPClient — no Firebase library needed.
- Added the Wi-Fi wait loop (manual starts sending before Wi-Fi connects) and a % moisture conversion.

Connections

From	To
Sensor module VCC	3V3
Sensor module GND	GND
Sensor module AO (analog out)	A0

Program

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>

const char* ssid = "Your_SSID";
const char* password = "Your_PASSWORD";

// Realtime Database URL from Firebase console (no trailing slash)
String FIREBASE_URL  = "https://your-project-default-rtdb.firebaseio.com";
String FIREBASE_AUTH = "";        // database secret; leave "" if rules are in test mode

const int sensorPin = A0;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
}

void loop() {
  int raw = analogRead(sensorPin);                // high = dry, low = wet
  int percent = map(raw, 1023, 300, 0, 100);      // calibrate 1023/300 for your sensor
  percent = constrain(percent, 0, 100);

  Serial.print("Raw: ");
  Serial.print(raw);
  Serial.print("  Moisture: ");
  Serial.print(percent);
  Serial.println(" %");

  WiFiClientSecure client;
  client.setInsecure();                           // skip certificate check (lab use)
  HTTPClient https;

  String url = FIREBASE_URL + "/SoilMoisture.json";
  if (FIREBASE_AUTH.length() > 0) url += "?auth=" + FIREBASE_AUTH;

  https.begin(client, url);
  https.addHeader("Content-Type", "application/json");
  String json = "{\"raw\":" + String(raw) + ",\"percent\":" + String(percent) + "}";
  int code = https.PUT(json);                     // PUT overwrites /SoilMoisture
  Serial.print("Firebase HTTP code: ");
  Serial.println(code);                           // 200 = success
```

```
  https.end();

  delay(5000);
}
```

How it works

- Two probes measure soil resistance: dry soil = high resistance = high reading (~1023); wet = low (~300).
- map() converts the raw value to a 0–100 % moisture figure. Calibrate: note reading in air (dry) and in water (wet) and put those two numbers in map().
- Firebase Realtime Database has a REST API: any path + '.json' is a URL. HTTP PUT writes data there; HTTPS is required, setInsecure() skips certificate checking (acceptable for a lab).
- Open Firebase console → Realtime Database → Data — the values update every 5 s.

Expected Serial Monitor output

```
WiFi connected
Raw: 742  Moisture: 39 %
Firebase HTTP code: 200
Raw: 365  Moisture: 90 %
Firebase HTTP code: 200
```

Firebase setup (5 minutes)

- console.firebase.google.com → Add project (disable Analytics) → Build → **Realtime Database** → Create database → **Start in test mode**.
- Copy the database URL from the top of the Data tab into FIREBASE_URL (no trailing slash).
- Optional secret: Project settings → Service accounts → Database secrets → Show.

If it doesn't work

- HTTP 401 → database rules don't allow writes: in Rules tab use test mode, or put the database secret in FIREBASE_AUTH.
- HTTP 404 / -1 → wrong URL. Copy it exactly from the Data tab. Non-US databases look like `https://name-default-rtdb.asia-southeast1.firebasedatabase.app`.
- Readings don't change → you used the DO (digital) pin instead of AO.

Viva — quick answers

What is Firebase?
Google's backend-as-a-service platform; the Realtime Database is a cloud-hosted NoSQL JSON tree synced to clients in real time.

Why is data stored as JSON?
Firebase RTDB is a JSON tree; each node is a key–value path like /SoilMoisture/percent.

PUT vs POST in Firebase?
PUT overwrites the data at that path; POST adds a new child with an auto-generated unique key (use it to keep a history).

How does a soil moisture sensor work?
It measures resistance/conductivity between two probes; water increases conductivity, lowering the output voltage.

What is a database secret?
A legacy authentication token that grants full access to the database (deprecated; production apps use Firebase Auth tokens).

Experiment 9 — Intrusion Detection System (Ultrasonic Sensor)

Aim: To build an intrusion detection system using Arduino and an HC-SR04 ultrasonic sensor.

Components: Arduino Uno, HC-SR04, LED, buzzer, jumper wires.

Changed from the manual

- Added a 30 ms timeout and a duration > 0 check — the original reported a false intruder when no echo came back.
- Added the LED and buzzer listed in the components.

Connections

From	To
HC-SR04 VCC / GND	5V / GND
HC-SR04 Trig	Pin 9
HC-SR04 Echo	Pin 10
Buzzer +	Pin 8 (– to GND)
LED	Pin 13 (on-board works)

Program

```
#define trigPin 9
#define echoPin 10
#define ledPin 13
#define buzzerPin 8

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
}

void loop() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);          // 10 us trigger pulse
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long duration = pulseIn(echoPin, HIGH, 30000);   // 30 ms timeout
  int distance = duration * 0.034 / 2;             // cm

  if (duration > 0 && distance < 20) {
    Serial.print("Intruder detected! Distance: ");
    Serial.print(distance);
    Serial.println(" cm");
    digitalWrite(ledPin, HIGH);
    digitalWrite(buzzerPin, HIGH);
  } else {
    Serial.println("Area clear.");
    digitalWrite(ledPin, LOW);
    digitalWrite(buzzerPin, LOW);
  }
  delay(500);
}
```

How it works

- A 10 µs HIGH on Trig makes the sensor send 8 pulses of 40 kHz ultrasound.
- Echo goes HIGH for the time sound takes to travel to the object and back. pulseIn() measures it in µs.
- Speed of sound ≈ 343 m/s = 0.034 cm/µs. Distance = time × 0.034 / 2 (÷2 because it's a round trip).
- Object closer than 20 cm → 'Intruder detected', LED and buzzer ON.

Expected Serial Monitor output

```
Area clear.
Area clear.
Intruder detected! Distance: 12 cm
Intruder detected! Distance: 9 cm
Area clear.
```

If it doesn't work

- Always 'Area clear' → Trig/Echo swapped.
- Distance jumps randomly → aim at a flat surface; soft objects (cloth) absorb sound.

Viva — quick answers

Working principle of HC-SR04?
Echo ranging (SONAR): measures time of flight of a 40 kHz ultrasonic pulse reflected from an object.

Range of HC-SR04?
About 2 cm to 400 cm, ~3 mm resolution, 15° beam angle.

Why divide by 2?
The pulse travels to the object and back, so the one-way distance is half.

What does pulseIn() do?
Waits for the pin to go HIGH and returns how long (in µs) it stayed HIGH; returns 0 on timeout.

Applications?
Security systems, parking sensors, obstacle-avoiding robots, tank level measurement.

Experiment 10 — Directional Control of DC Motor (L298N)

Aim: To control the direction of rotation of a DC motor using Arduino and an L298N motor driver.

Components: Arduino Uno, L298N motor driver module, DC motor, 9–12 V battery, jumper wires.

Changed from the manual

- Added a 1 s stop between directions (reversing instantly stresses the motor and driver) and Serial messages so the examiner sees the state.

Connections

From	To
L298N IN1 / IN2	Pin 8 / Pin 9
L298N ENA	Leave the jumper ON (motor A always enabled, full speed)
L298N OUT1 / OUT2	Motor terminals
L298N 12V terminal	Battery +
L298N GND	Battery − **and** Arduino GND (common ground is essential)

Program

```
int in1 = 8;
int in2 = 9;

void setup() {
  Serial.begin(9600);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
}

void loop() {
  // Clockwise
  digitalWrite(in1, HIGH);
  digitalWrite(in2, LOW);
  Serial.println("Clockwise");
  delay(2000);

  // Stop (protects motor and driver before reversing)
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  Serial.println("Stop");
  delay(1000);

  // Anti-clockwise
  digitalWrite(in1, LOW);
  digitalWrite(in2, HIGH);
  Serial.println("Anti-clockwise");
  delay(2000);

  // Stop
  digitalWrite(in1, LOW);
  digitalWrite(in2, LOW);
  Serial.println("Stop");
  delay(1000);
}
```

How it works

- The L298N is a dual H-bridge: four switches that can apply the supply across the motor in either polarity.
- IN1=HIGH, IN2=LOW → clockwise. IN1=LOW, IN2=HIGH → anti-clockwise. Both LOW → stop (both HIGH → brake).
- Speed control (if asked): remove the ENA jumper, wire ENA to PWM pin 10 and use `analogWrite(10, 0–255)`.

Expected Serial Monitor output

```
Clockwise
Stop
Anti-clockwise
Stop
```

If it doesn't work

- Motor doesn't move but driver LED is on → no common ground, or battery too weak (L298N drops ~2 V).
- Never power the motor from the Arduino 5V pin.

Viva — quick answers

What is an H-bridge?
A circuit of four switches arranged like an 'H' that lets current flow through the load in either direction, reversing the motor.

Why not drive the motor directly from Arduino?
Motors draw hundreds of mA and create back-EMF spikes; Arduino pins give ~20 mA.

Role of ENA pin?
Enable for motor A; a PWM signal on it controls speed.

What is PWM?
Pulse Width Modulation: rapidly switching ON/OFF; the duty cycle sets the average voltage and hence speed.

Experiment 11 — Distance Measurement to Google Cloud

Aim: To measure distance using an ultrasonic sensor and post the value to a Google cloud platform.

Components: NodeMCU ESP8266, HC-SR04, 1 kΩ and 2 kΩ resistors, jumper wires.

Changed from the manual

- **The manual only has placeholder code**, and the service it names (Google Cloud IoT Core) no longer exists. This is a complete, working program that posts to Firebase.

Connections

From	To
HC-SR04 VCC	VIN (5 V)
HC-SR04 GND	GND
HC-SR04 Trig	D5 (GPIO14) — 3.3 V trigger works fine
HC-SR04 Echo	Echo → 1 kΩ → D6 (GPIO12); D6 → 2 kΩ → GND (drops 5 V echo to 3.3 V)

Program

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClientSecure.h>

const char* ssid = "Your_SSID";
const char* password = "Your_PASSWORD";
String FIREBASE_URL  = "https://your-project-default-rtdb.firebaseio.com";
String FIREBASE_AUTH = "";

const int trigPin = D5;     // GPIO14
const int echoPin = D6;     // GPIO12 (through 1k/2k divider from 5 V echo)

float readDistance() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  long duration = pulseIn(echoPin, HIGH, 30000);
  return duration * 0.034 / 2;               // cm (0 = no echo)
}

void setup() {
  Serial.begin(115200);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");
}

void loop() {
  float distance = readDistance();
  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");

  WiFiClientSecure client;
  client.setInsecure();
  HTTPClient https;
  String url = FIREBASE_URL + "/Distance.json";
  if (FIREBASE_AUTH.length() > 0) url += "?auth=" + FIREBASE_AUTH;

  https.begin(client, url);
```

```
  https.addHeader("Content-Type", "application/json");
  int code = https.PUT(String(distance));      // stores the number at /Distance
  Serial.print("Cloud HTTP code: ");
  Serial.println(code);
  https.end();

  delay(5000);
}
```

How it works

- Distance is measured exactly as in Exp 9; then the number is written with an HTTPS PUT to `/Distance.json` in Firebase.
- Firebase is part of Google Cloud, so this satisfies 'Google cloud IoT platform'. Reuse the database you made in Exp 8.

Expected Serial Monitor output

```
WiFi connected
Distance: 23.45 cm
Cloud HTTP code: 200
Distance: 11.02 cm
Cloud HTTP code: 200
```

If it doesn't work

- Distance always 0 → sensor not getting 5 V (use VIN, not 3V3), or Echo divider wired wrong.
- Same Firebase errors as Exp 8 (401 rules, 404 URL).

Viva — quick answers

What was Google Cloud IoT Core?
Google's managed MQTT/HTTP device bridge with device registry; Google retired it on 16 August 2023, so new projects use Firebase, Pub/Sub or third-party platforms.

Why a voltage divider on Echo?
HC-SR04 outputs 5 V; ESP8266 GPIO is 3.3 V and not officially 5 V tolerant.

HTTP vs MQTT for sending sensor data?
HTTP: request–response, heavier headers, simple and universal. MQTT: persistent connection, tiny headers, push-based pub/sub, better for frequent small messages.

Experiment 12 — Simple Sensor Application — DHT11 Temperature & Humidity

Aim: To develop a simple sensor-based application using DHT11 with Arduino.

Components: Arduino Uno, DHT11 (3-pin module), jumper wires.

> **Changed from the manual**
> - Added a failed-read check so you don't get 'nan' printed. Other easy 'simple sensor applications' if the examiner wants something different: the LDR (Exp 3) or ultrasonic (Exp 9) programs.

Connections

From	To
DHT11 VCC (+)	5V
DHT11 GND (−)	GND
DHT11 DATA (out)	Pin 7 (a bare 4-pin sensor needs a 10 kΩ pull-up from DATA to 5V; modules have it built in)

Program

```
#include <DHT.h>
#define DHTPIN 7
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  delay(2000);                        // DHT11 needs ~1-2 s between reads
  float h = dht.readHumidity();
  float t = dht.readTemperature();    // Celsius

  if (isnan(h) || isnan(t)) {
    Serial.println("Failed to read from DHT11 sensor!");
    return;
  }
  Serial.print("Temp: ");
  Serial.print(t);
  Serial.print(" C   Humidity: ");
  Serial.print(h);
  Serial.println(" %");
}
```

How it works

- DHT11 has a capacitive humidity sensor and an NTC thermistor plus a small chip that sends a 40-bit digital packet on a single wire.
- The library handles the timing; readHumidity() and readTemperature() return floats, or NaN if the read failed.
- Ranges: 0–50 °C (±2 °C), 20–90 % RH (±5 %). One read per ~1–2 s maximum.

Expected Serial Monitor output

```
Temp: 28.00 C   Humidity: 62.00 %
Temp: 28.00 C   Humidity: 61.00 %
```

If it doesn't work

- 'Failed to read' every time → wrong pin, missing pull-up, or you selected DHT22 instead of DHT11.
- 'DHT.h: No such file' → install 'DHT sensor library' by Adafruit (accept the prompt to also install Adafruit Unified Sensor).

Viva — quick answers

Is DHT11 analog or digital?
Digital — it sends data on a single-wire serial protocol.

DHT11 vs DHT22?
DHT22 is more accurate (±0.5 °C, ±2 % RH), wider range (−40 to 80 °C, 0–100 % RH), slower and costlier.

What does isnan() check?
Whether the reading is 'Not a Number', which the library returns when the sensor didn't respond.

Applications?
Weather stations, greenhouse monitoring, HVAC control, server room monitoring.

Exam-day checklist

Writing the record / answer sheet (standard order)

- Aim → Components required (table) → Circuit/connection diagram (draw neatly with pin labels) → Procedure (4–6 steps) → Program → Output/observation → Result.
- Result line format: '*The [sensor/device] was interfaced with Arduino/NodeMCU and [what happened] was successfully observed.*'
- Draw the pin-to-pin connections exactly as in the tables in this guide — examiners check pin numbers against your code.

Executing

- Wire with the board **unplugged**. Double-check 5 V vs 3.3 V before plugging in.
- Select the right board and port before uploading; open Serial Monitor and set the baud.
- For cloud experiments, set up the ThingSpeak channel / Firebase database **tonight** and save the keys and URL in a note — creating accounts in the exam wastes time and may hit college Wi-Fi restrictions.
- Carry: phone hotspot (2.4 GHz), a data USB cable, and this guide.

Formulas to memorise

What	Formula
Uno ADC → volts	V = reading × 5.0 / 1024
NodeMCU ADC → volts	V = reading × 3.3 / 1023
LM35	°C = V × 100 (10 mV/°C)
Ultrasonic distance	cm = duration(µs) × 0.034 / 2
LED resistor	R = (5 − 2) / 0.015 ≈ 220 Ω
Voltage divider	Vout = Vin × R2 / (R1 + R2)